Artifact Appendix: Vista: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing

Submission ID #217

1 Artifact Overview

This artifact supports the CAIS 2026 paper "Vista: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing." The artifact contains the training code for Vista, the staged OpenQASM verifier used during agentic reinforcement learning, generation scripts for Vista and baseline checkpoints, and offline evaluation scripts for reproducing the reported semantic metrics.

The main artifact components are grouped by role:

- **Training and RL orchestration.** `examples/train/quantum/` contains the GRPO/FSDP training scripts for the 4B quantum circuit model.
- **Verifier and reward implementation.** `verl_tool/servers/tools/quantum_cpu.py` provides the quantum verifier service used during tool-augmented rollouts, and `verl_tool/servers/tools/utils/quantum_reward_cal.py` implements staged rewards for syntax feasibility, behavior similarity, expectation-value objective quality, and utility-oriented optimization.
- **Model-output generation.** `quantum-code-generation/code/generation/` contains scripts for generating OpenQASM circuits from Vista and baseline checkpoints.
- **Offline evaluation and metric recomputation.** `quantum-code-generation/code/evaluation/` parses generated QASM, simulates circuits, computes relative entropy, checks expectation values, and writes summary statistics; `artifact/scripts/run_quantum_experiment.sh` is the reviewer wrapper for generation plus evaluation or evaluation of an existing raw generation file.
- **Dataset construction and packaged inputs.** `quantum-code-generation/code/data_generation/src/algorithms/` contains graph-generation source files and included pickle inputs for the supported graph optimization primitives.
- **Figure and table reproduction.** `artifact/scripts/draw_vista_figures.py` and `vista_draw/` provide headless wrappers and plot-ready inputs for regenerating Figures; `artifact/tables/` and `artifact/scripts/show_paper_tables.py` provide extracted CSV values for Tables 1–3 and the figure/table map.
- **Released checkpoints and public data.** The artifact references the Hugging Face RL checkpoint `Benyucong/rl_quantum_4b`, SFT initialization checkpoint `Benyucong/sft_quantum_circuit_gen_4B`, and dataset `Benyucong/graph-data-quantum-rl`.

2 Badge Target

We request the Functional and Results Reproduced badges for a GitHub-only submission at https://github.com/benyucong/rl-quantum. CAIS's Available checklist requires a public archive with irrevocable versioning and long-term storage, such as Zenodo but not GitHub, so we do not rely on the Available badge unless a DOI-backed archive is added later. The repository includes public checkpoint and dataset links, and the final submitted version should include raw generated-circuit JSON files for all reported model runs plus precomputed logs/results.

3 Hardware and Software Requirements

The minimal artifact inspection path requires Python 3.10 or 3.11 and does not require a GPU. Recomputing evaluation metrics from raw generated circuits requires the Python packages listed in `quantum-code-generation/code/evaluation/requirements.txt`, including Qiskit 1.2.4, qiskit-aer 0.16.1, and qiskit-qasm3-import 0.5.1.

Regenerating model outputs requires a GPU-capable machine with enough memory for the selected 3B/4B model checkpoint. Full Vista retraining is substantially more expensive: the paper-scale configuration fine-tunes a 4B model with GRPO and FSDP on 8 AMD MI250X GPUs or 8 NVIDIA H100 GPUs. The artifact also includes smaller single-node Slurm wrappers intended for sanity checks or partial reproduction.

4 Major Claims and Reproduction Plan

Claim	Result and action
End-to-end quality	**Reported result:** Reported Table 1 gives Vista Pass@1 SCR 99.31%, SREV 22.41%, RE 11.61, HQCR 17.24%; Pass@10 SCR 100%, SREV 33.10%, RE 8.48, HQCR 27.24%. **Reviewer action:** Recompute `summary_stats_*.json` from raw generation JSONs with `evaluate_samples.py`.
Reward ablation	**Reported result:** Reported Table 2 gives full Vista as best overall and removing RE as the largest drop. **Reviewer action:** Evaluate the full and ablation raw outputs with the same evaluator.
Verifier efficiency	**Reported result:** Figures 7–9 report verifier time drops from 8.20s to 4.63s per iteration, a 1.77x speedup. **Reviewer action:** Inspect timing logs or rerun gated/non-gated variants under the same budget.
Scaling and robustness	**Reported result:** Figures 2, 4, and 5 report stronger results across thresholds, graph primitives, and 8–16 qubit sizes. **Reviewer action:** Redraw with `draw_vista_figures.py`.
Hardware deployment	**Reported result:** Figures 10–14 report preserved objective quality with shorter scheduled circuit duration on real/emulated QPU backends. **Reviewer action:** Inspect archived hardware logs or rerun postprocessing if QPU access is available.

5 Evaluation Workflow

Quickest check. This CPU-only check validates that the artifact is present and that included metric/table files can be parsed:

```
cd /path/to/rl-quantum
python3 artifact/scripts/summarize_eval_outputs.py \
```

```
  quantum-code-generation/code/evaluation/out
python3 artifact/scripts/show_paper_tables.py all
```

It prints a summary of included precomputed evaluation files and the extracted Tables 1–3.

Fresh GPU run. This path generates new Vista samples, evaluates them, and draws the plots directly supported by fresh evaluator output:

```
bash artifact/scripts/run_quantum_experiment.sh \
  --n-samples 50 \
  --out-dir artifact_runs/reviewer_smoke
EVAL_DIR=artifact_runs/reviewer_smoke/evaluation
FIG_DIR=artifact_runs/reviewer_smoke/figures
python3 artifact/scripts/draw_vista_figures.py \
  --evaluation-dir "$EVAL_DIR" \
  --only box,relative_entropy \
  --output-dir "$FIG_DIR" \
  --strict
```

The wrapper writes raw generations plus `summary_stats_VISTA.json`/`summary_VISTA_raw_data.csv`. The plotting wrapper converts that CSV into objective-gap and relative-entropy plot inputs.

All packaged figures. To redraw all packaged Figures from `vista_draw` plot-ready CSV/JSON inputs and logs, run:

```
python3 artifact/scripts/draw_vista_figures.py \
  --input-dir vista_draw \
  --output-dir artifact_runs/paper_figures \
  --strict
```

This runs the packaged plot tasks for objective gap, compilability, relative entropy, scalability, per-primitive breakdown, training dynamics, verifier efficiency, stage cost, latency, hardware reward stability, and real-device quality–efficiency trade-off.

Table 1 maps Figures to the artifact tasks used by this wrapper. The complete script/input map is in `artifact/tables/figure_artifact_map.csv`.

Table 1: Figure-to-artifact task correspondence.

Item	Artifact task	Main evidence
Fig. 1	`design_trace`	Training, verifier, and reward code inspection
Fig. 2	`per_primitive`	`per_primitive_hqcr.csv`
Fig. 3	`relative_entropy`, `box`	`relative_entropy_table.csv`, `box.csv`
Fig. 4	`scalability_qubits`	`scalability_table_0.3.csv`
Fig. 5	`scalability_gates_depth`	Gates/depth scalability CSVs
Fig. 6	`training_dynamics`	Training-dynamics CSVs
Fig. 7	`verifier_efficiency`	Verifier-efficiency CSV
Fig. 8	`training_logs`	Budget-matched outputs from `logs.csv`
Fig. 9	`stage_cost`	Verifier timing from `logs.csv`
Fig. 10	`helmi_reward_stability`, `real_device_tradeoff`	Archived hardware/QPU report JSON
Fig. 11	`latency_breakdown`	Archived hardware/QPU report JSON
Fig. 12	`helmi_reward_stability`	Archived hardware/QPU report JSON
Fig. 13	`training_logs`	`budget_matched_q5_*` outputs
Fig. 14	`real_device_tradeoff`	Hardware quality–efficiency table/report
Fig. 15	`dataset_trace`	Graph-generation pickles and Table 3

Table reproduction. The `show_paper_tables.py` command above is display-only. To reproduce Tables 1–2 from raw generated outputs, rerun the evaluator:

```
cd quantum-code-generation/code/evaluation
python3 -m venv .venv-eval
source .venv-eval/bin/activate
pip install -r requirements.txt
GEN_JSON=../generation/out/<RAW_GENERATION_JSON>
python3 src/evaluate_samples.py \
  "$GEN_JSON" \
  out \
  <MODEL>
```

The command writes `summary_stats_<MODEL>.json` and `summary_<MODEL>_raw_data.csv`. Compare these outputs with the extracted Table CSVs in `artifact/tables/`.

Model-output regeneration. Evaluators with GPU access can regenerate model outputs from the released checkpoint:

```
cd quantum-code-generation/code/generation
python3 -m venv .venv-gen
source .venv-gen/bin/activate
pip install -r requirements.txt
python3 generate_samples.py \
  --uid cais26 \
  --model_path Benyucong/rl_quantum_4b \
  --dataset Benyucong/graph-data-quantum-rl \
  --n_samples 200
```

6 Notes for the AEC

Full retraining is not expected for kick-the-tires because it requires a large GPU cluster. The intended reproducibility path is to use the released checkpoint and raw generation outputs, then rerun the deterministic evaluation scripts. Real-QPU experiments depend on external hardware availability and should be evaluated from archived hardware logs unless the AEC explicitly requests a live run.

The current artifact includes graph-generation pickle inputs, precomputed summary JSON/CSV files, the public RL checkpoint `Benyucong/rl_quantum_4b`, the public SFT checkpoint `Benyucong/sft_quantum_circuit_gen_4B`, and the public dataset `Benyucong/graph-data-quantum-rl`. For a low-friction Results Reproduced evaluation, the final archive should also include the raw generated-circuit JSON files used as inputs to `evaluate_samples.py`.

The submitted artifact URL is https://github.com/benyucong/rl-quantum. If a DOI-backed archive is added later, list that archive URL in HotCRP as well.